Introduction

Dr. RAJIB MALL

Professor

Department Of Computer Science & Engineering

IIT Kharagpur.

About The Instructor

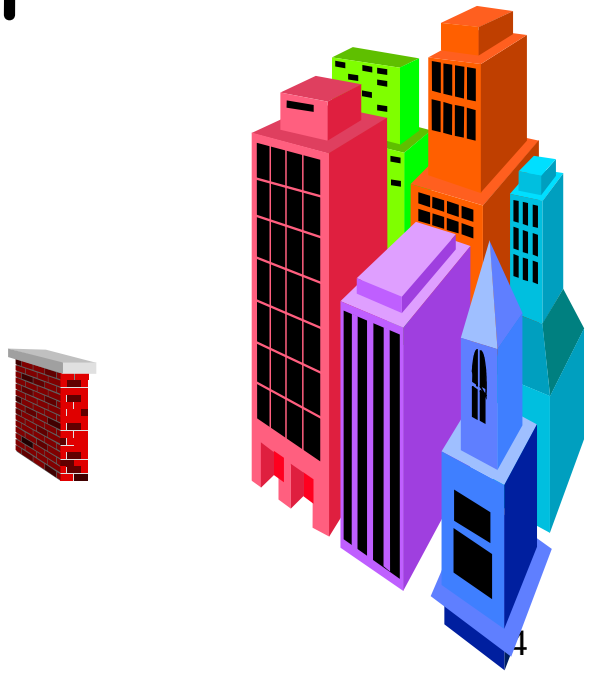
- RAJIB MALL
- B.E. , M.E., Ph.D from Indian Institute of Science, Bangalore
- Worked with Motorola (India)
- Shifted to IIT, Kharagpur in 1994
  - Currently Professor and HOD

Review Questions

- What is the difference between abstraction and decomposition?
- How are these two concepts useful in software engineering?

What is Software Engineering?

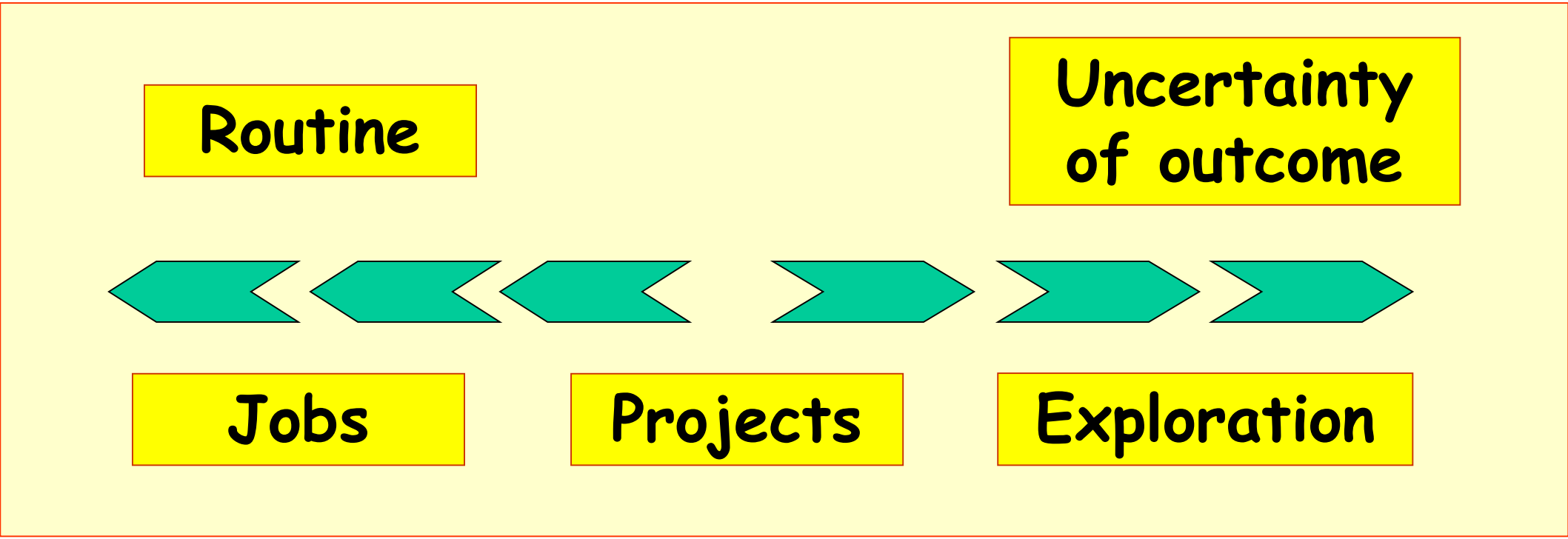
- Engineering approach to develop software.
  - Building Construction Analogy.
- Systematic collection of past experience:
  - Techniques,
  - Methodologies,
  - Guidelines.



Techniques versus Methodology…

- Many times used interchangeably…
- **Dictionary:**
  - **Technique:** a systematic procedure, formula, or a routine by which a task is accomplished.
  - **Method:** A habitual, logical, or prescribed practice or a preordained sequence of achieving certain end results with accuracy and efficiency.
    - However, when a method is systematic and based upon logic, it is referred to as a scientific method.

Jobs versus Projects?



Jobs – repetition of very well-defined and well understood tasks with very little uncertainty

Exploration – The outcome is very uncertain, e.g. finding a cure for cancer.

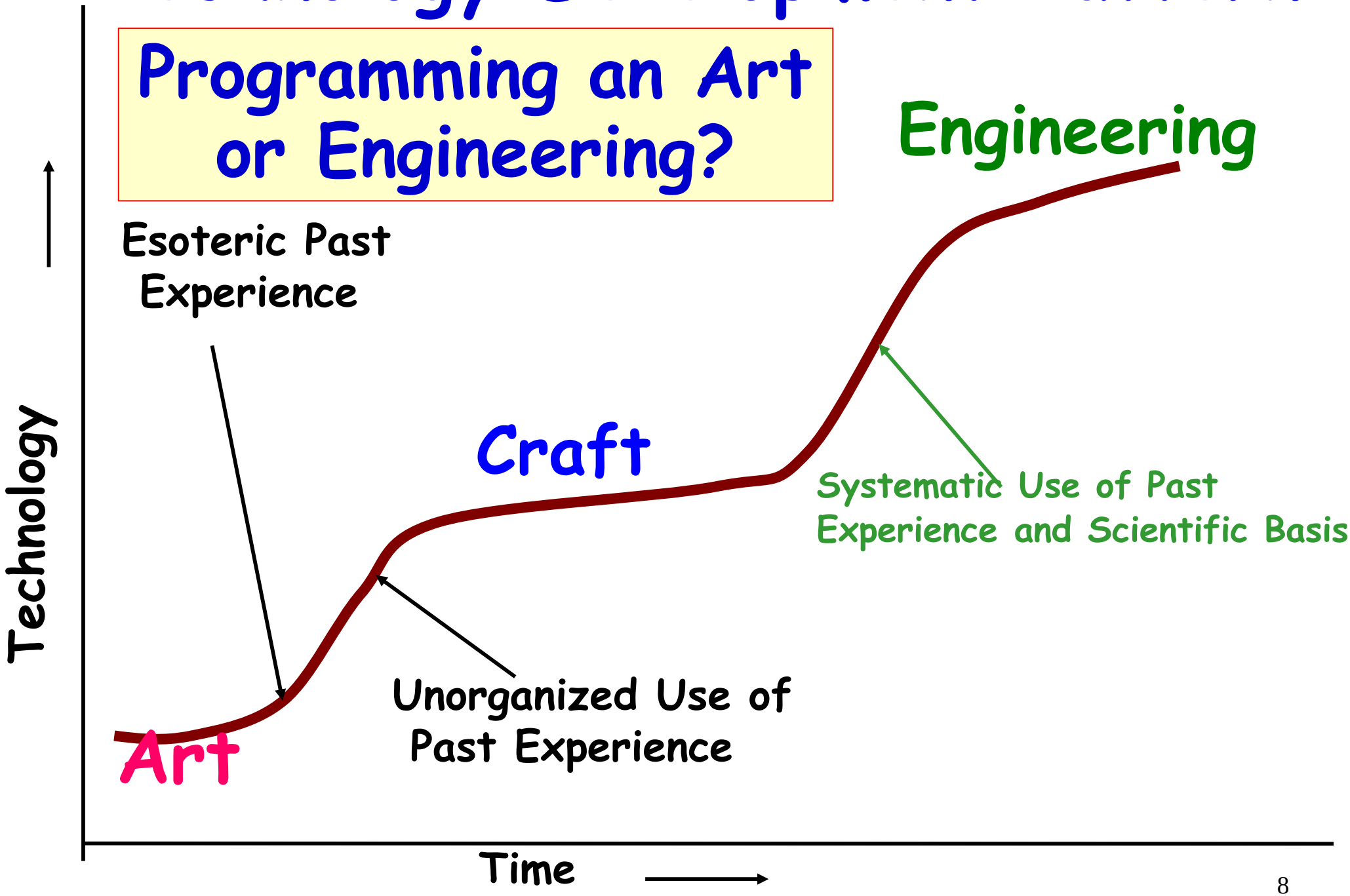
Projects – in the middle! Has challenge as well as routine…

IEEE Definition

- "Software engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software."

Technology Development Pattern

Programming an Art or Engineering?



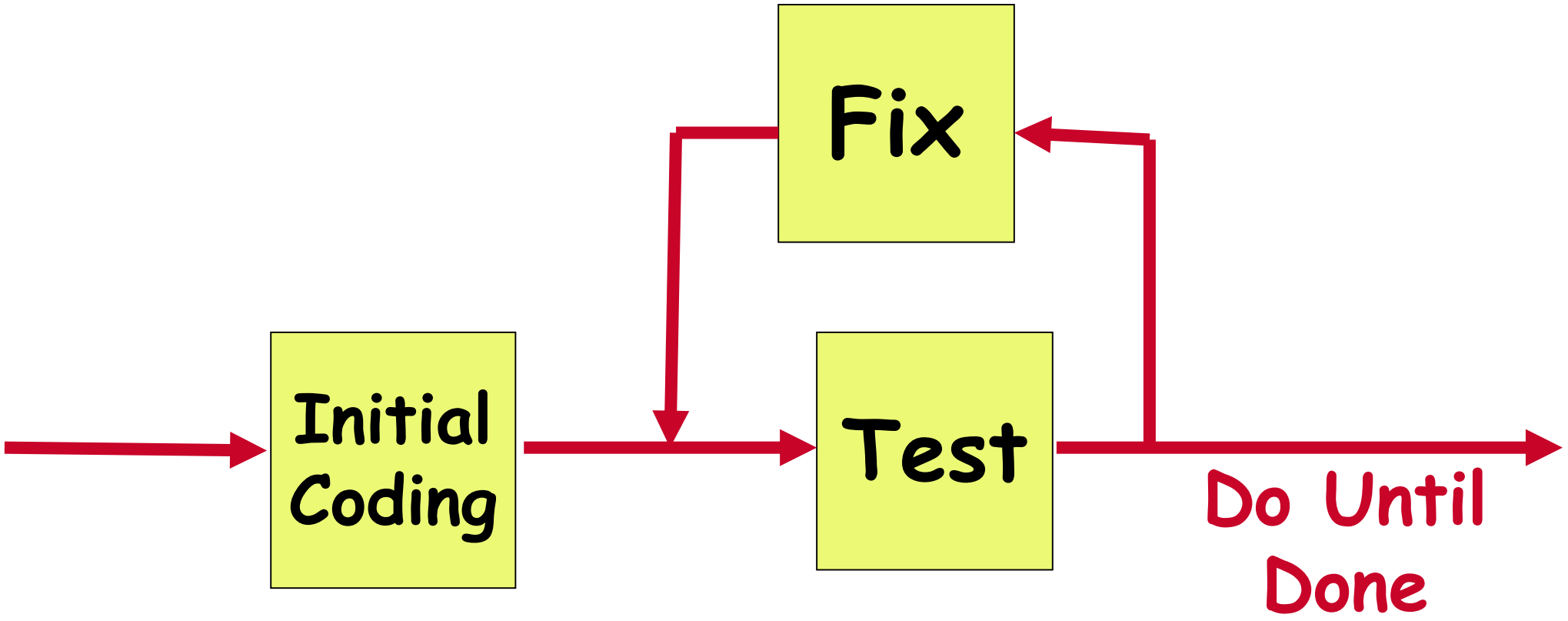
Programming an Art or Engineering?

- Heavy use of past experience:
  - Past experience is systematically arranged.
- Theoretical basis and quantitative techniques provided.
- Many are just thumb rules.
- Tradeoff between alternatives.
- Pragmatic approach to cost-effectiveness.

What is Exploratory Development?

- Early programmers used an exploratory (also called build and fix) style.
  - A `dirty' program is quickly developed.
  - The bugs are fixed as and when they are noticed.
  - Similar to a junior student develops programs…

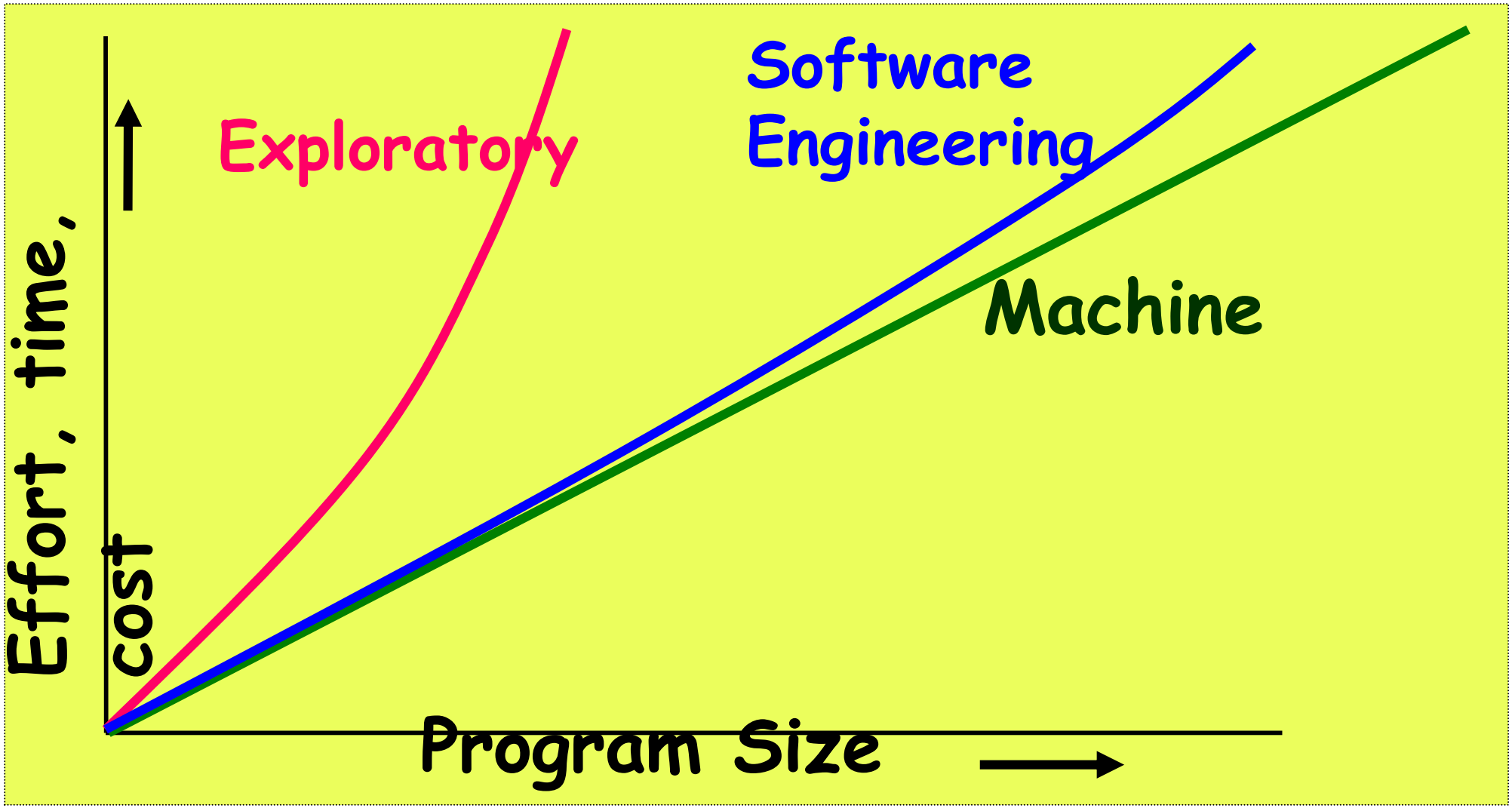
Exploratory Style

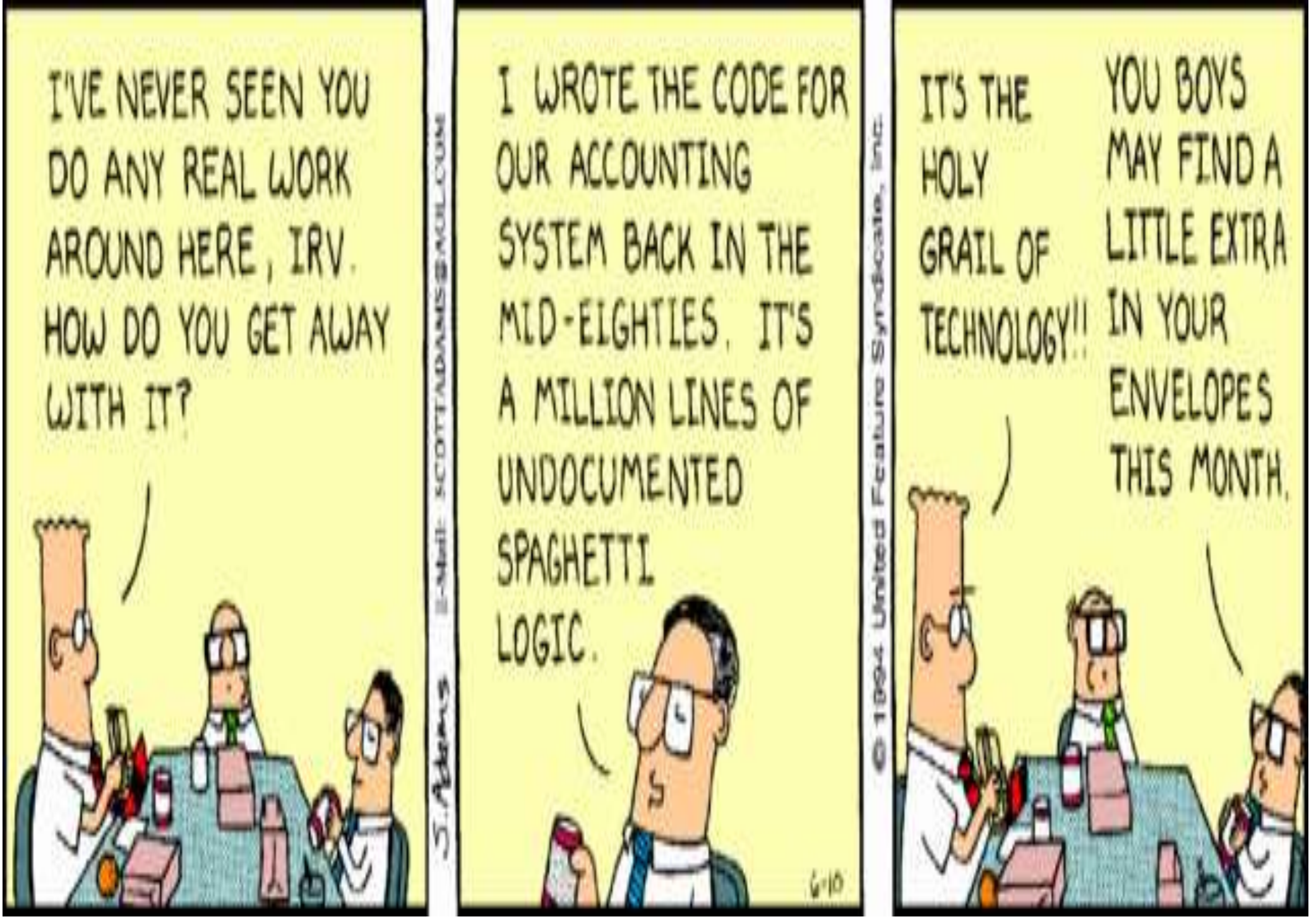


Does not work for nontrivial projects…
Why?...

What is Wrong with the Exploratory Style?

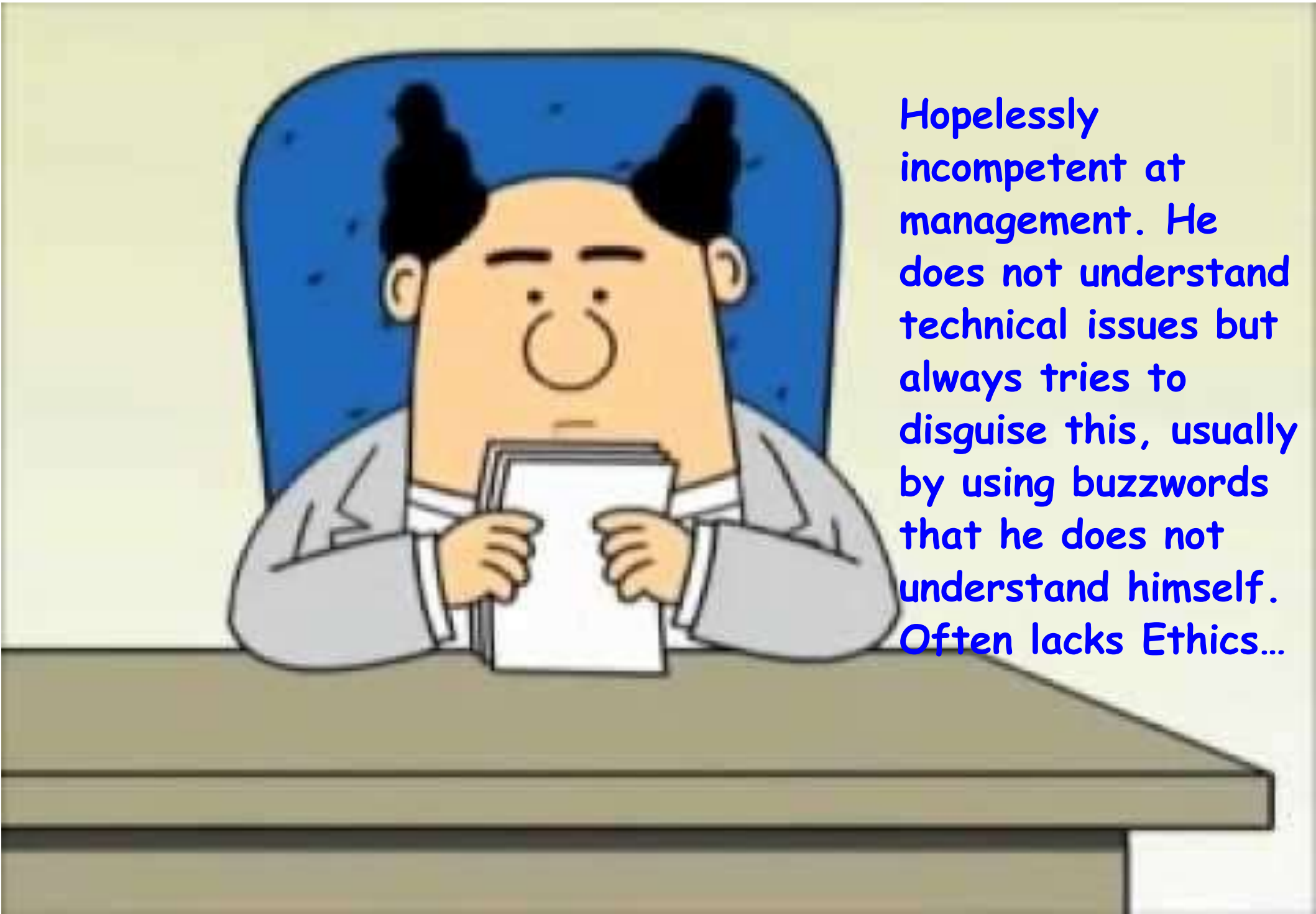
- Can successfully be used for developing very small (toy) programs.





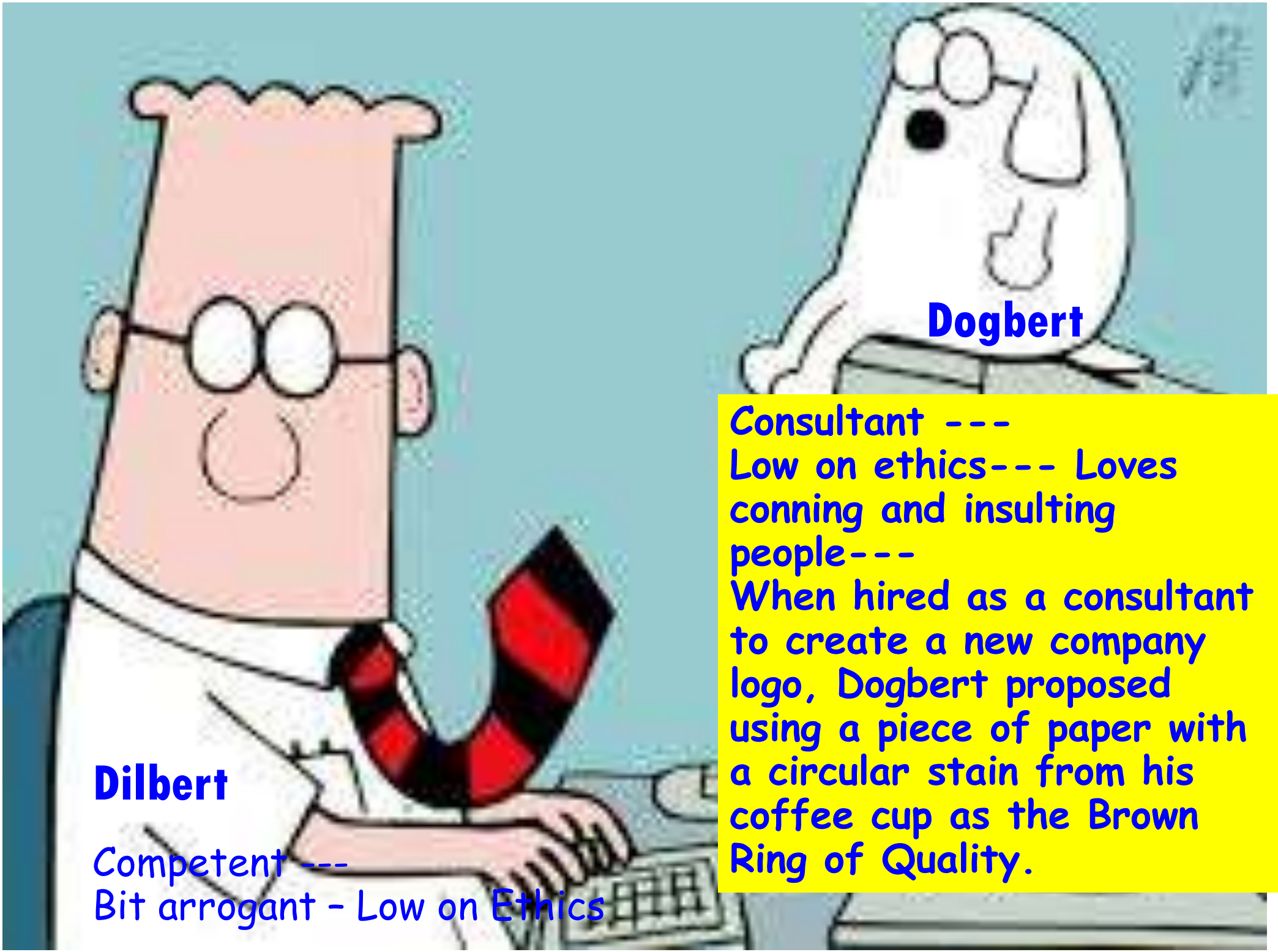
Dilbert: Scott Adams

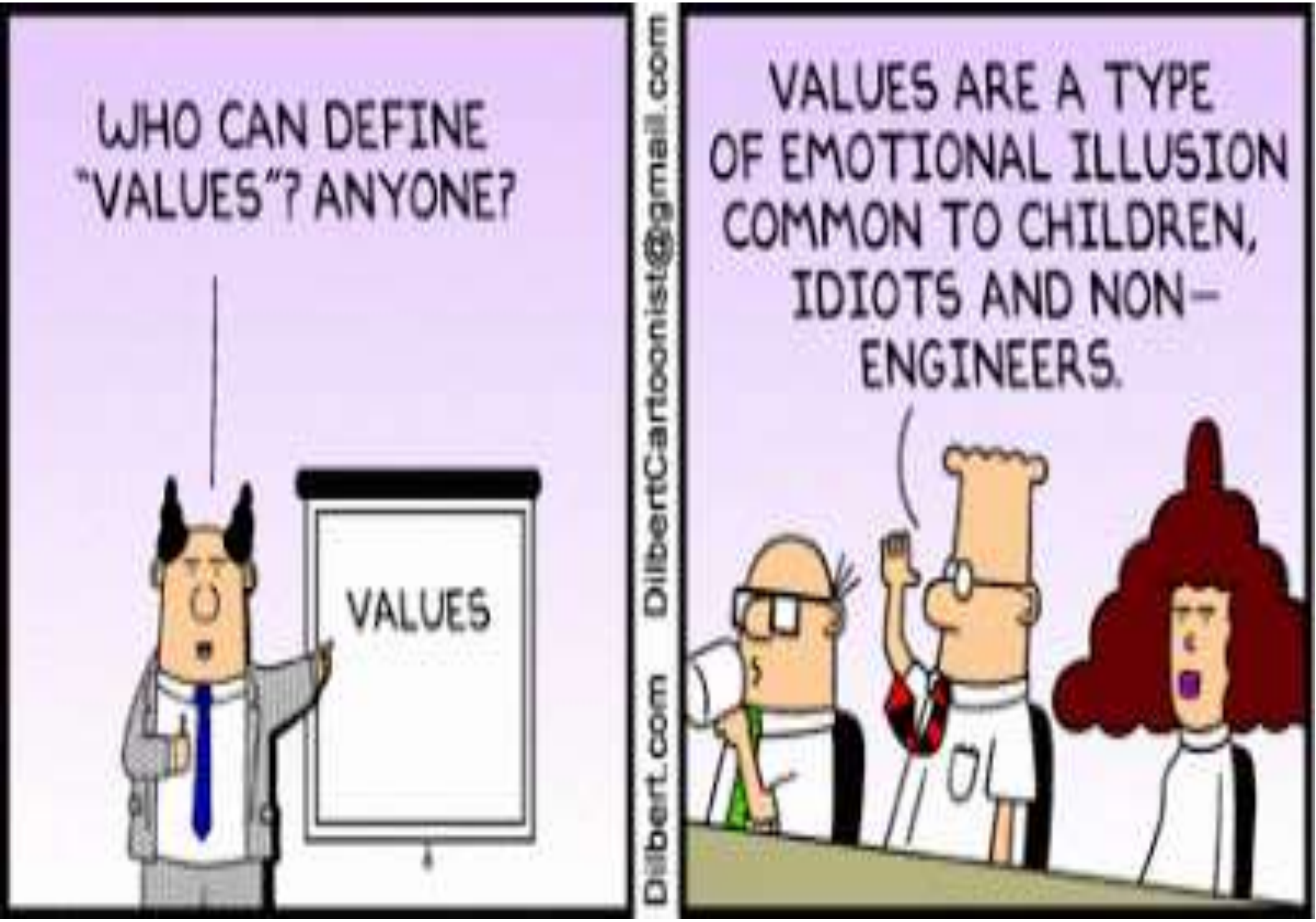
Our Project Manager: Ponty Haired Boss



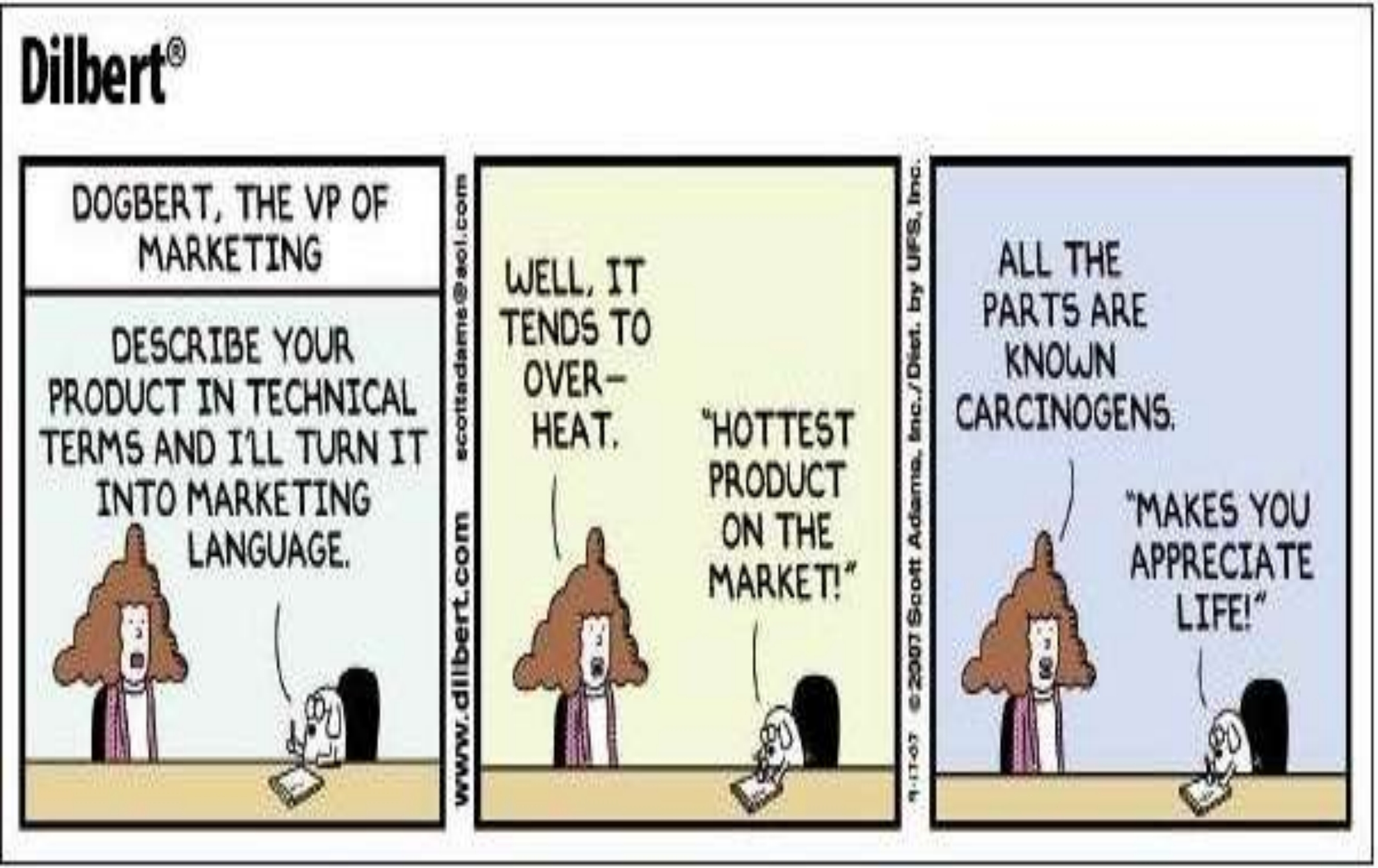
Hopelessly incompetent at management. He does not understand technical issues but always tries to disguise this, usually by using buzzwords that he does not understand himself. Often lacks Ethics…

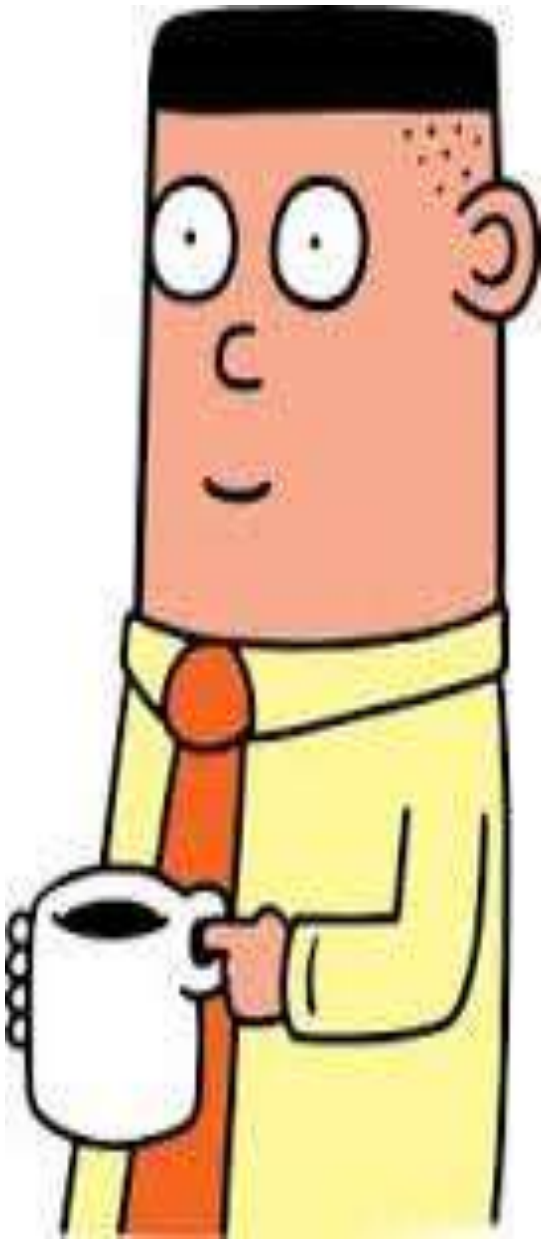

MANAGER TRAINING
YOU WILL OFTEN BE ASKED TO COMMENT ON THINGS YOU DON'T UNDERSTAND.
?
S. Adams E-mail: SCOTTADAMS@AOL.COM
THESE HANDOUTS CONTAIN NONSENSE PHRASES THAT CAN BE USED IN ANY SITUATION.
BUZZWORD GENERATOR
11-18-97 © 1997 United Feature Syndicate, Inc.
... SO, LET'S DOMINATE OUR INDUSTRY ... WITH QUALITY IMPLEMENTATION OF METHODOLOGIES.
I'LL GET RIGHT ON IT.


Dogbert
Consultant ---
Low on ethics--- Loves conning and insulting people---
When hired as a consultant to create a new company logo, Dogbert proposed using a piece of paper with a circular stain from his coffee cup as the Brown Ring of Quality.
Dilbert
Competent ---
Bit arrogant – Low on Ethics

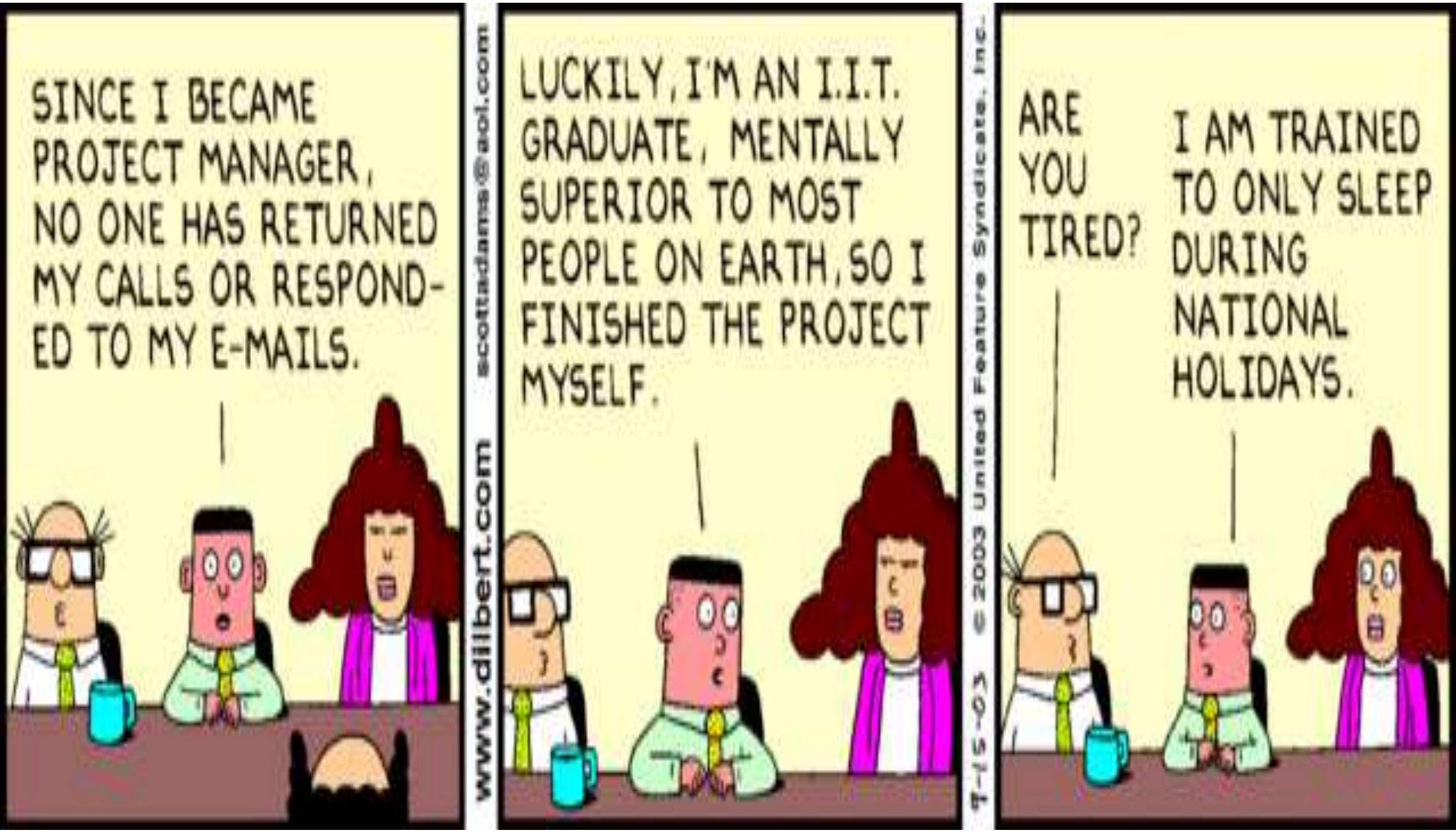

WHO CAN DEFINE "VALUES"? ANYONE?
VALUES
Dilbert.com DilbertCartoonist@gmail.com
VALUES ARE A TYPE OF EMOTIONAL ILLUSION COMMON TO CHILDREN, IDIOTS AND NON-ENGINEERS.


OUR CONSULTANT WILL TELL US HOW WE CAN SECURE A LONG-TERM SUPPLY OF RARE EARTH METALS FOR OUR PRODUCTS.
DilbertCartoonist@gmail.com
Dilbert.com
CHINA HAS MOST OF THE RARE EARTH METALS. TRY DYING. AND REINCARNATING. THERE'S A 20% CHANCE THAT YOU'LL BE BORN CHINESE.
2-28-11 ©2011 Scott Adams, Inc./Dist. by UFS, Inc.
WHAT'S PLAN B?
IF THE ONLY PART THAT GOES WRONG IS THE CHINESE PART, YOU CAN TRY DYING AGAIN.


Dilbert®
DOGBERT, THE VP OF MARKETING
DESCRIBE YOUR PRODUCT IN TECHNICAL TERMS AND I'LL TURN IT INTO MARKETING LANGUAGE.
www.dilbert.com scottadams@aol.com
WELL, IT TENDS TO OVER-HEAT.
"HOTTEST PRODUCT ON THE MARKET!"
9-17-07 ©2007 Scott Adams, Inc./Dist. by UFS, Inc.
ALL THE PARTS ARE KNOWN CARCINOGENS.
"MAKES YOU APPRECIATE LIFE!"



Asok graduated from the Indian Institute of Technology (IIT). Asok's test scores (a perfect 1600 on the old SAT) and the fact his IQ is 240, show that he is the smartest member of the engineering team. He is often called upon by the Boss to do odd jobs and his ideas are usually left hanging in the meetings. There are a few jokes about him having psychic powers which he learned at the IIT. Yet despite his intelligence, ethics and mystical powers, Asok sometimes takes advice from Wally in the arts of laziness, and Dilbert in surviving the office.


SINCE I BECAME PROJECT MANAGER, NO ONE HAS RETURNED MY CALLS OR RESPONDED TO MY E-MAILS.
www.dilbert.com scottadams@aol.com
LUCKILY, I'M AN I.I.T. GRADUATE, MENTALLY SUPERIOR TO MOST PEOPLE ON EARTH, SO I FINISHED THE PROJECT MYSELF.
ARE YOU TIRED?
I AM TRAINED TO ONLY SLEEP DURING NATIONAL HOLIDAYS.
9-15-03 © 2003 United Feature Syndicate, Inc.


BEWARE THE POWER OF STINK EYE, INTERN. I WILL MAKE YOU BOW TO MY WILL!
GAAA!!!
www.dilbert.com scottadams@aol.com
MUST... USE... BANNED TELEKINETIC POWERS TO NEUTRALIZE THREAT.
GRRRR!!!
YOU HAVE A CALL FROM THE INDIAN INSTITUTE OF TECHNOLOGY. IT'S SOMEONE FROM THE DEPARTMENT OF THINGS YOU SHOULDN'T DO.
© 2008 Scott Adams, Inc./Dist. by UFS, Inc.

What is Wrong with the Exploratory Style?

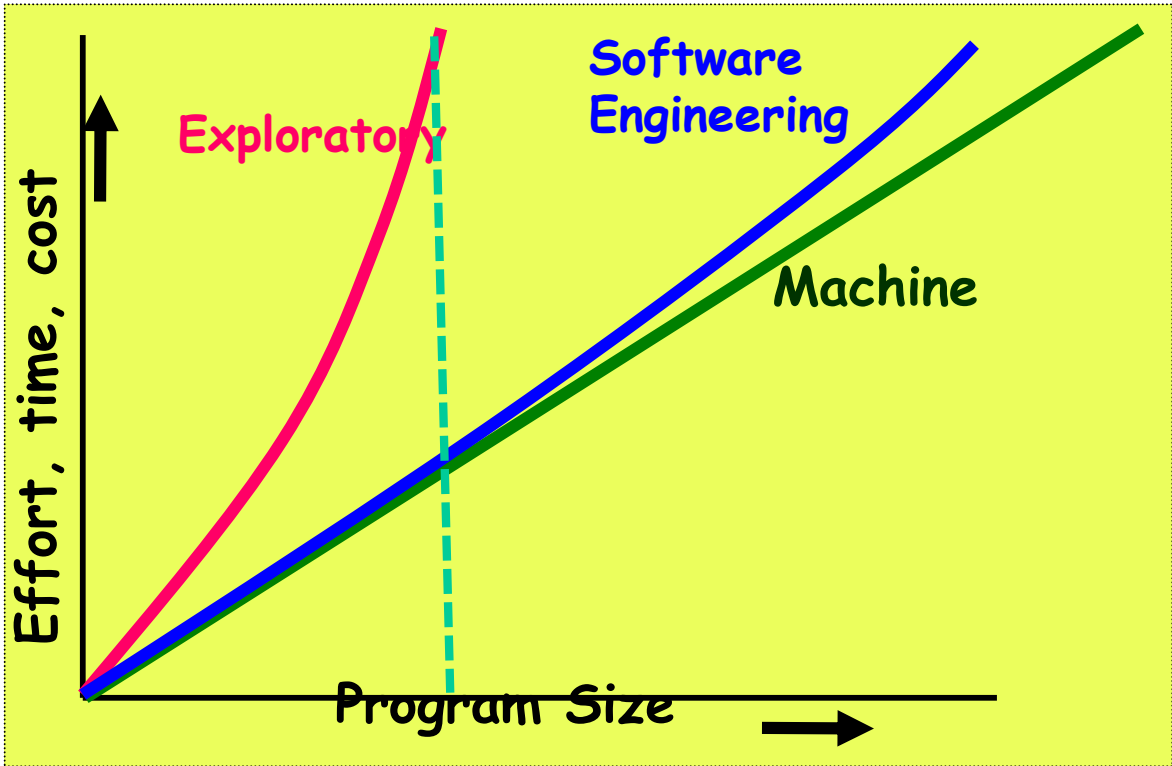
Cont…

- Besides the exponential growth of effort, cost, and time with problem size:
  - Exploratory style usually results in unmaintenable code.
  - **It becomes very difficult to use the exploratory style in a team development environment…**

What is Wrong with the Exploratory Style?

Cont…

- Why does the effort required to develop a software grow exponentially with size?
  - Why does the approach completely breaks down when the size of required software becomes large?



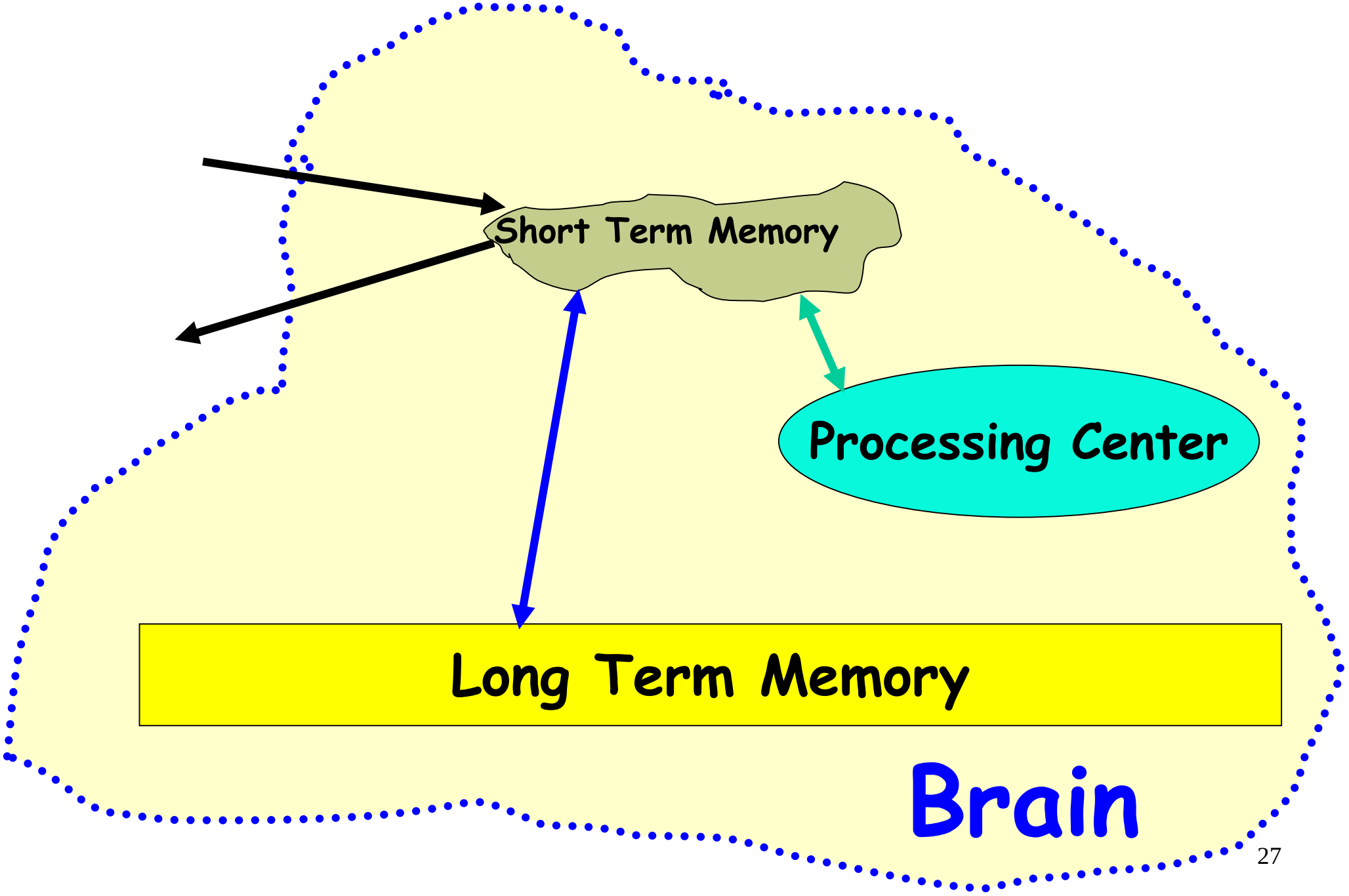
An Interpretation Based on Human Cognition Mechanism

- Human memory can be thought to be made up of two distinct parts [Miller 56]:
  - **Short term memory and**
  - **Long term memory.**

Human Cognition Mechanism

- Suppose I ask: **"If it is 10:10AM now, how many hours are remaining today?"**
  - 10AM would be stored in the short-term memory.
  - "A day is 24 hours long." would be fetched from the long term memory into short term memory.
  - The mental manipulation unit would compute the difference (24-10).

Schematic Representation of Brain



Short Term Memory

- An item stored in the short term memory can get lost:
  - Either due to decay with time or
  - Displacement by newer information.
- This restricts the time for which an item is stored in short term memory:
  - Typically few tens of seconds.
  - However, an item can be retained longer in the short term memory by recycling.

What is an Item?

- An item is any set of related information.
  - A character such as `a' or a digit such as `5'.
  - A word, a sentence, a story, or even a picture.
- Each item normally occupies one place in memory.
- When you are able to relate several different items together (chunking):
  - The information that should normally occupy several places, takes only one place in memory.

Chunking

- If you are given the binary number 110010101001
  - It may prove very hard for you to understand and remember.
  - But, the octal form of 6251 (110)(010)(101)(001) would be easier.
  - You have managed to create chunks of three items each.

Evidence of Short Term Memory

- In many of our day-to-day experiences:
  - **Short term memory is evident.**
- Suppose, you look up a number from the telephone directory and start dialling it.
  - If you find the number is busy, you can dial the number again after a few seconds without having to look up the directory.
- But, after several days:
  - You may not remember the number at all
  - Would need to consult the directory again.

The Magical Number 7

- If a person deals with seven or less number of items:
  - These would be accommodated in the short term memory.
  - So, he can easily understand it.
- As the number of new information increases beyond seven:
  - It becomes exceedingly difficult to understand it.

What is the Implication in Program Development?

- A small program having just a few variables:
  - Is within easy grasp of an individual.
- As the number of independent variables in the program increases:
  - It quickly exceeds the grasping power of an individual…
  - **Requires an unduly large effort to master the problem.**

Implication in Program Development

- Instead of a human, if a machine could be writing (generating) a program,
  - The slope of the curve would be linear.
- But, how does use of software engineering principles helps hold down the effort-size curve to be almost linear?
  - **Software engineering principles extensively use techniques specifically targeted to overcome the human cognitive limitations.**

Which Principles Deployed by Software Engineering Techniques to Overcome Human Cognitive Limitations?

- Two important principles are profusely used:
  - Abstraction
  - Decomposition

What is Abstraction?

- Simplify a problem by omitting unnecessary details.
  - **Focus attention on only one aspect of the problem and ignore other aspects and irrelevant details.**
  - **Also called model building.**

Questions

- What is a model?
- Why develop a model? That is, how does a model help?
- Give some examples of models.

Abstraction Example

- Suppose you are asked to develop an overall understanding of some country.
  - Would you:
    - Meet all the citizens of the country, visit every house, and examine every tree of the country?
  - You would possibly refer to various types of maps for that country only.

You would study an Abstraction…

- A map is:
  - An abstract representation of a country.
  - Various types of maps (abstractions) possible.

Does a Problem have a single Abstraction?

- Several abstractions of the same problem can be created:
  - Focus on some specific aspect and ignore the rest.
  - Different types of models help understand different aspects of the problem.

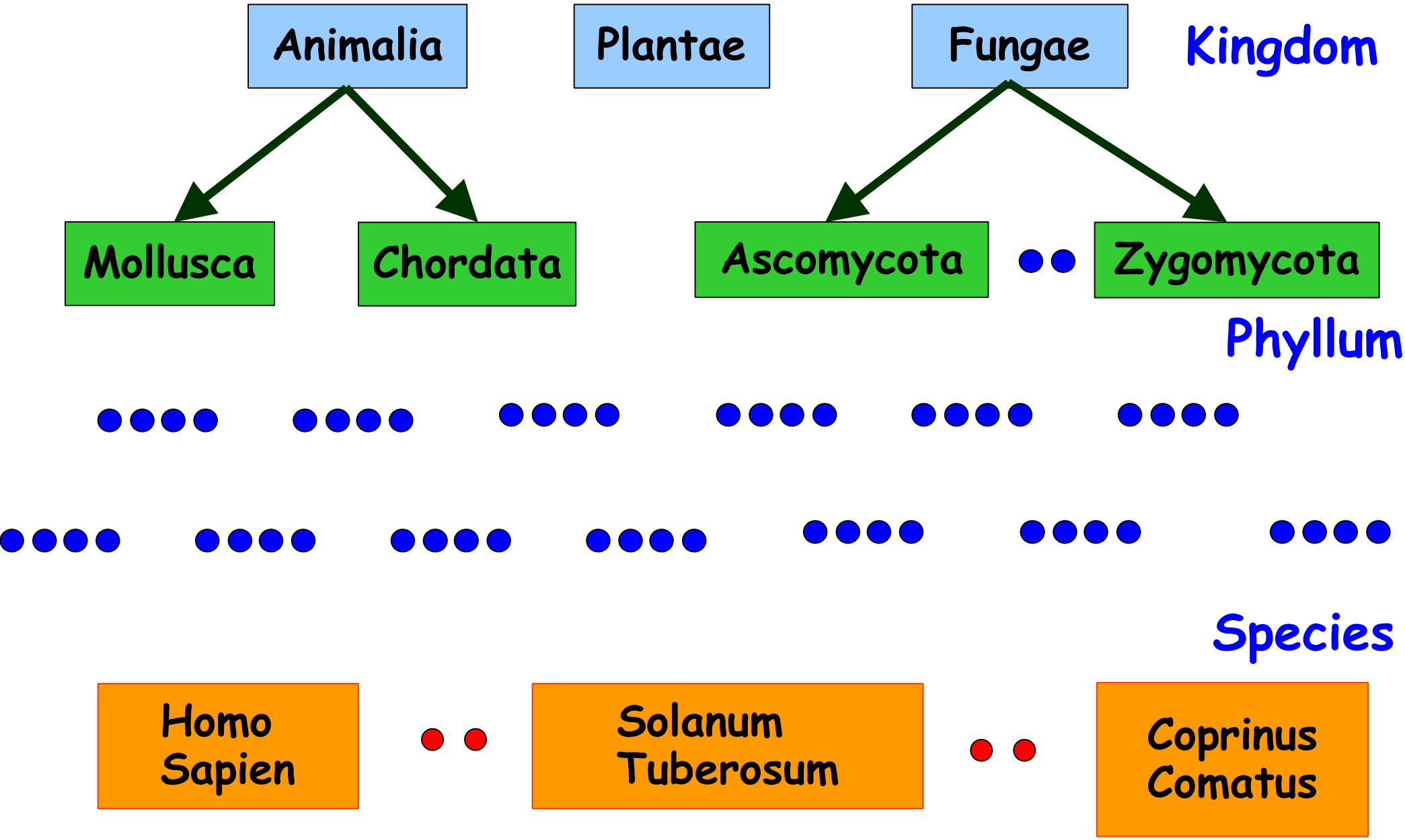
Abstractions of Complex Problems

- For complex problems:
  - A single level of abstraction is inadequate.
  - A hierarchy of abstractions may have to be constructed.
- Hierarchy of models:
  - A model in one layer is an abstraction of the lower layer model.
  - An implementation of the model at the higher layer.

Complex Problem Abstraction Example

- Suppose you are asked to understand all life forms that inhabit the earth.
- Would you start examining each living organism?
  - You will almost never complete it.
  - Also, get thoroughly confused.
- **Solution**: You can try to build an abstraction hierarchy.

Living Organisms



Decomposition

- Decompose a problem into many small independent parts.
  - The small parts are then taken up one by one and solved separately.
  - The idea is that each small part would be easy to grasp and therefore can be easily solved.
  - The full problem is solved when all the parts are solved.

Decomposition

- A popular use of decomposition principle:

  - Try to break a bunch of sticks tied together versus breaking them individually.

- Any arbitrary decomposition of a problem may not help.

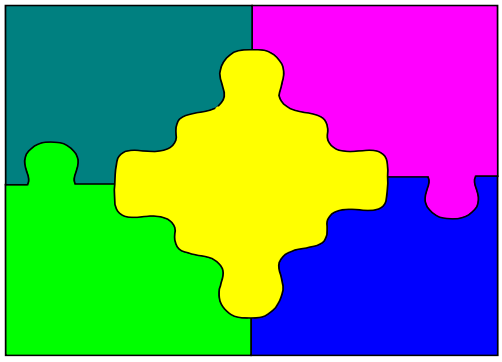
  - The decomposed parts must be more or less independent of each other.

Decomposition Example

- Example use of decomposition principle:
  - You understand a book better when the contents are organized into independent chapters.
  - Compared to when everything is mixed up.

Why Study Software Engineering? (1)

- To acquire skills to develop large programs.
  - Handling exponential growth in complexity with size.
  - Systematic techniques based on abstraction (modelling) and decomposition.



Why Study Software Engineering? (2)

- Learn systematic techniques of:
  - Specification, design, user interface development, testing, project management, etc.
  - Appreciate issues that arise in team development.

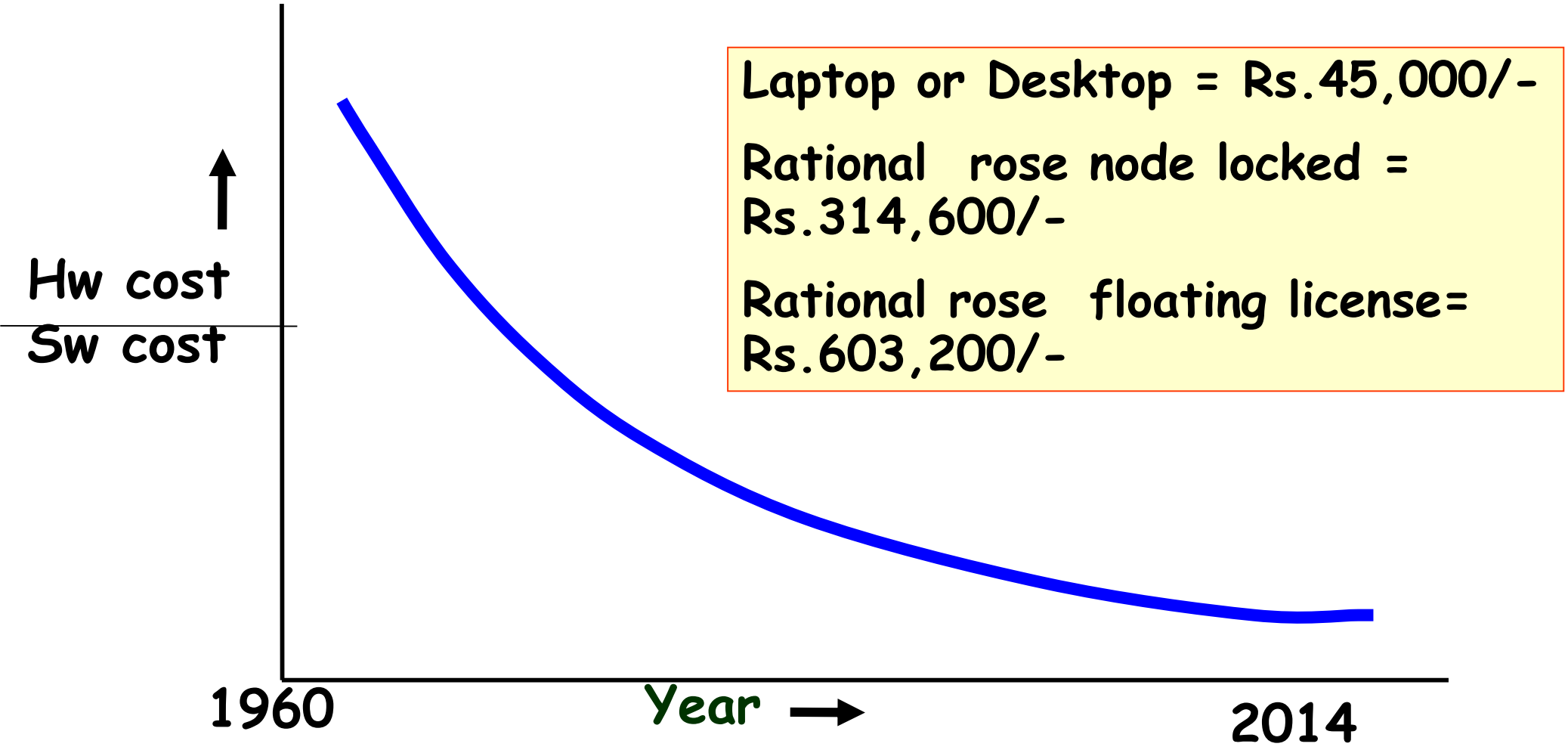
Why Study Software Engineering? (3)

- To acquire skills to be a better programmer:
  - Higher Productivity
  - Better Quality Programs

Software Crisis

- It is often the case that software products:
  - Fail to meet user requirements.
  - Expensive.
  - Difficult to alter, debug, and enhance.
  - Often delivered late.
  - Use resources non-optimally.

Software Crisis (cont.)



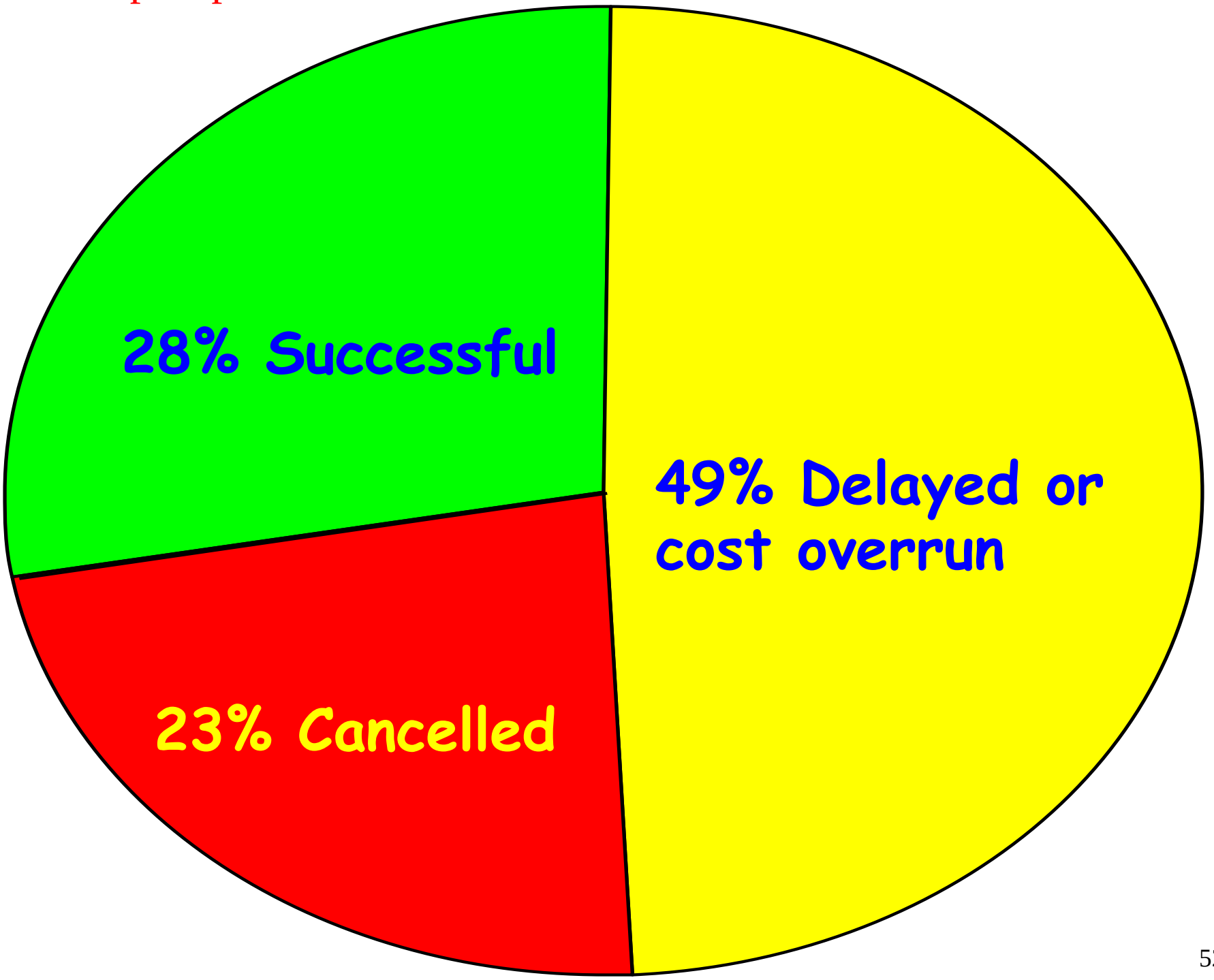
Laptop or Desktop = Rs.45,000/-

Rational rose node locked = Rs.314,600/-

Rational rose floating license= Rs.603,200/-

Relative Cost of Hardware and Software

Standish Group Report



Then why not have entirely hardware systems?

- A virtue of software:
  - Relatively easy to change
  - Otherwise it might as well be hardware
- The more the complexity of a software (Entropy), the harder it is to change--why?
  - Further, the more the changes made to a program, the greater is the increase in its entropy.

Which Factors are Contributing to the Software Crisis?

- Larger problems,
- Poor project management
- Lack of adequate training in software engineering,
- Increasing skill shortage,
- Low productivity improvements.

Programs versus Software Products

• Usually small in size	• Large
• Author himself is sole user	• Large number of users
• Single developer	• Team of developers
• Lacks proper user interface	• Well-designed interface
• Lacks proper documentation	• Well documented & user-manual prepared
• Ad hoc development.	• Systematic development

Types of Software Projects

- Two types of software projects:
  - Products (Generic software)
  - Services (custom software)
- Total business - appx. $1 Trillion
  - Half in products and half services
  - **Services segment is growing fast!**

Types of Software

Packaged software—prewritten software available for purchase

- **Horizontal market software**—meets needs of many companies
- **Vertical market software**—designed for particular industry

Custom software—software developed at user's request

Custom software-Developer tailors his generic solution

Types of Software Projects

- Software product development projects
- Software services projects

Software Services

- Software service is an umbrella term, includes:
  - **Software customization**
  - **Software maintenance**
  - **Software testing**
  - **Also contract programmers who carry out coding or any other assigned activities.**

Scenario of Indian Software Companies

- Indian companies have focused on the services segment --- Why?

A Few Changes in Software Project Characteristics over Last 40 Years

- 40 years back, very few software existed
  - **Every project started from scratch**
  - **Projects were multi year long**
- The programming languages that were used hardly provided any scope for reuse:
  - FORTRAN, PASCAL, COBOL, BASIC
- No application was GUI-based:
  - Mostly command selection from displayed text menu items.

Traditional versus Modern Projects

- Projects are increasingly becoming services:
  - Either tailoring some existing software or reusing certain pre-built libraries.
- Facilitation and accommodation of client feedbacks
- Facilitation of customer participation in project development work
- Incremental software delivery with evolving functionalities.
- **No software being developed from scratch -- - Significant reuse**

Computer Systems Engineering

- Computer systems engineering:
  - encompasses software engineering.
- Many products require development of software as well as specific hardware to run it:
  - a coffee vending machine,
  - a sophisticated toy,
  - A new smart phone, etc.

Computer Systems Engineering

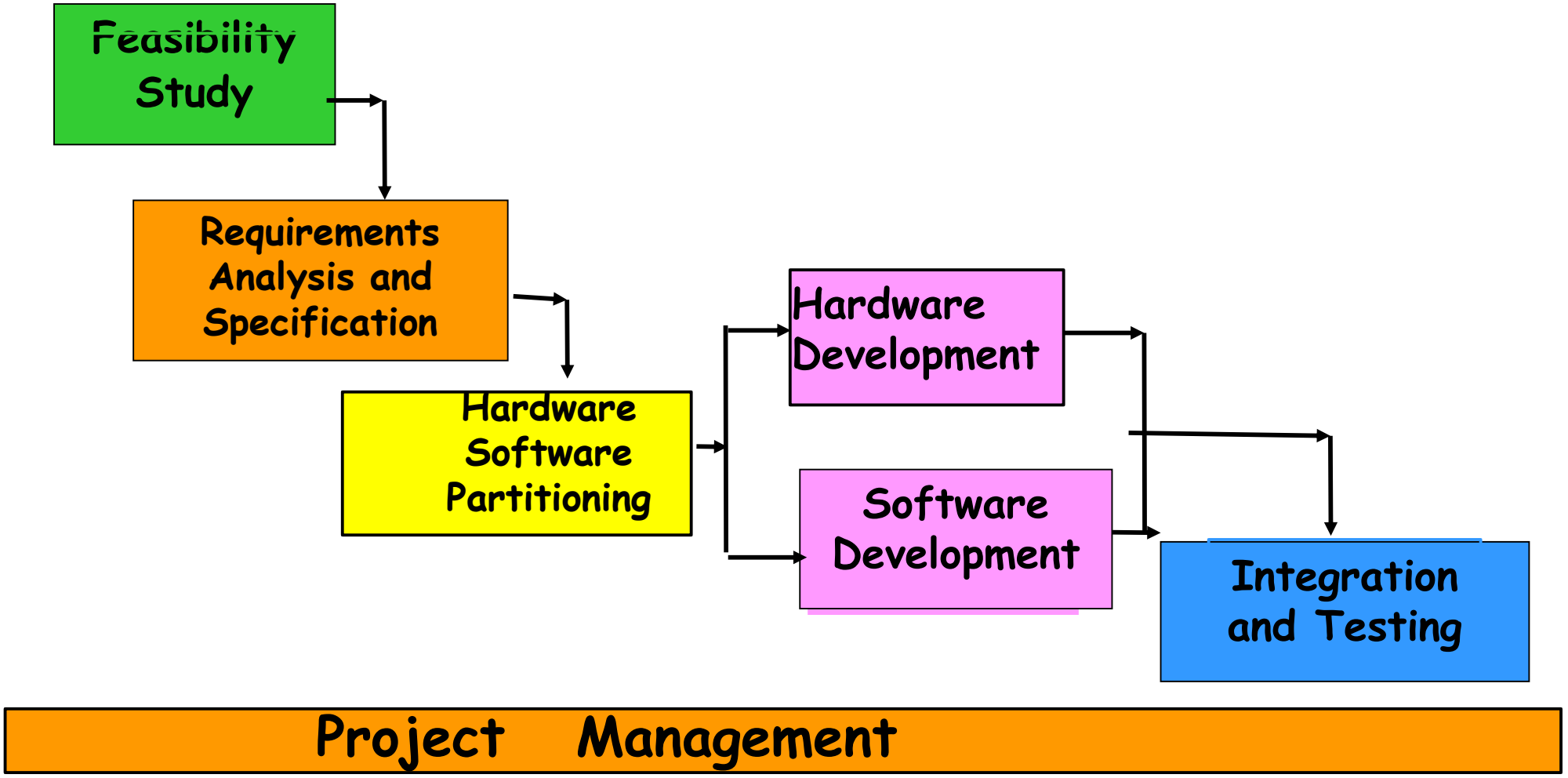
- The high-level problem:
  - Deciding which tasks are to be solved by software.
  - Which ones by hardware.

Computer Systems Engineering (CONT.)

- Typically, hardware and software are developed together:
  - Hardware simulator is used during software development.
- Integration of hardware and software.
- Final system testing

Computer Systems Engineering

(CONT.)



Emergence of Software Engineering

- Early Computer Programming (1950s):
  - Programs were being written in assembly language…
  - Sizes limited to about a few hundreds of lines of assembly code…

Early Computer Programming (50s)

- Every programmer developed his/her own style of writing programs:
  - According to his intuition (exploratory programming).

High-Level Language Programming
(Early 60s)

- High-level languages such as FORTRAN, ALGOL, and COBOL were introduced:
  - **This reduced software development efforts greatly.**
  - **Why reduces?**

High-Level Language Programming
(Early 60s)

- Software development style was still exploratory.
  - Typical program sizes were limited to a few thousands of lines of source code.

Control Flow-Based Design (late 60s)

- Size and complexity of programs increased further:
  - Exploratory programming style proved to be insufficient.
- Programmers found:
  - Very difficult to write cost-effective and correct programs.

Control Flow-Based Design (late 60s)

- Programmers found it very difficult:
  - To understand and maintain programs written by others.
- To cope up with this problem, experienced programmers advised--
  --"Pay particular attention to the design of the program's control structure."

Control Flow-Based Design (late 60s)

- A program's control structure indicates:
  - The sequence in which the program's instructions are executed.
- To help design programs having good control structure:
  - Flow charting technique was developed.

Control Flow-Based Design (late 60s)

- Using flow charting technique:
  - One can represent and design a program's control structure.
  - Usually one understands a program:
    - By mentally simulating the program's execution sequence.

Control Flow-Based Design

(Late 60s)

- A program having a messy flow chart representation:
  - Difficult to understand and debug.

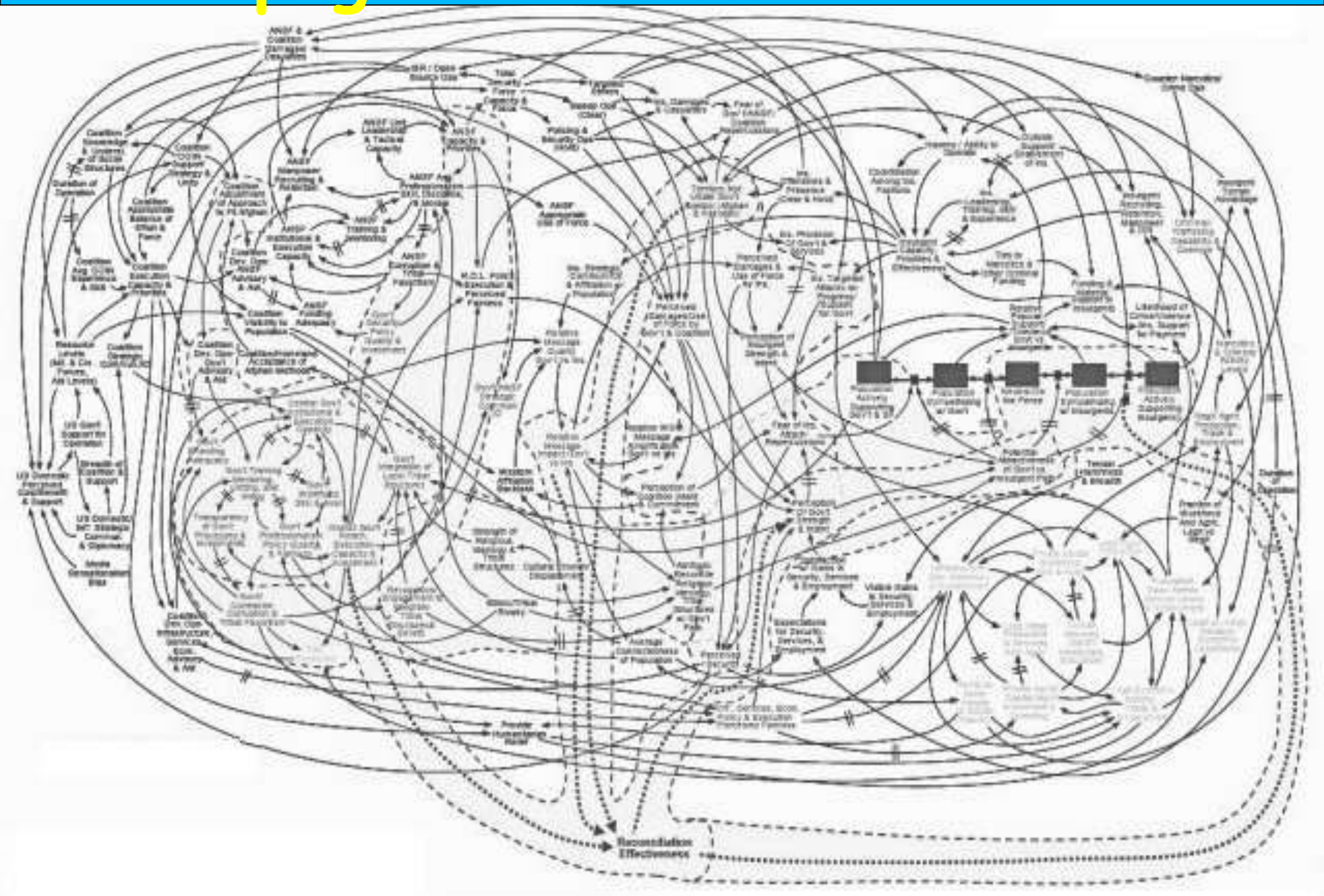
Control Flow-Based Design (Late 60s)

- It was found:
  - GO TO statements makes control structure of a program messy.
  - GO TO statements alter the flow of control arbitrarily.
  - The need to restrict use of GO TO statements was recognized.

Control Flow-Based Design (Late 60s)

- Many programmers had extensively used assembly languages.
  - JUMP instructions are frequently used for program branching in assembly languages.
  - Programmers considered use of GO TO statements inevitable.

Spaghetti Code Structure



Control-flow Based Design (Late 60s)

- At that time, Dijkstra published his article:
  - "Goto Statement Considered Harmful" Comm. of ACM, 1969.
- Many programmers were unhappy to read his article.

Control Flow-Based Design (Late 60s)

- They published several counter articles:
  - Highlighted the advantages and inevitability of GO TO statements.

Control Flow-Based Design (Late 60s)

- It soon was conclusively proved:
  - Only three programming constructs are sufficient to express any programming logic:
    - **sequence** (a=0;b=5;)
    - **selection** (if(c==true) k=5 else m=5;)
    - **iteration** (e.g. while(k>0) k=j-k;)

Control-flow Based Design (Late 60s)

- Everyone accepted:
  - It is possible to solve any programming problem without using GO TO statements.
  - This formed the basis of Structured Programming methodology.

Structured Programming

- A program is called structured:
  - When it uses only the following types of constructs:
    - sequence,
    - selection,
    - iteration

Structured Programs

- Unstructured control flows are avoided.
- Consist of modules.
- Use single-entry, single-exit program constructs.

Structured Programs

- However, violations to this feature are permitted:
  - Due to practical considerations such as:
  - Premature loop exit (break) to support exception handling.

Structured programs

- Structured programs are:
  - Easier to read and understand,
  - Easier to maintain,
  - Require less effort and time for development.

Structured Programming

- Research experience shows:
  - Programmers commit less number of errors:
    - While using structured if-then-else and do-while statements.
    - Compared to test-and-branch constructs.

Data Structure-Oriented Design
(Early 70s)

- Soon it was discovered:
  - **It is important to pay more attention to the design of data structures of a program**
    - Than to the design of its control structure.

Data Structure-Oriented Design
(Early 70s)

- Techniques which emphasize designing the data structure:
  - Derive program structure from it:
    - Are called **data structure-oriented design techniques.**

Data Structure Oriented Design (Early 70s)

- Example of data structure-oriented design technique:
  - Jackson's Structured Programming(JSP) methodology
    - Developed by Michael Jackson in 1970s.

Data Structure Oriented Design (Early 70s)

- JSP technique:
  - Program code structure should correspond to the data structure.

Data Structure Oriented Design
(Early 70s)

- JSP methodology:
  - A program's data structures are first designed using notations for
    - sequence, selection, and iteration.
  - Then data structure design is used :
    - To derive the program structure.

Data Structure Oriented Design
(Early 70s)

- Several other data structure-oriented Methodologies also exist:
  - e.g., Warnier-Orr Methodology.

Data Flow-Oriented Design

(Late 70s)

- Data flow-oriented techniques advocate:
  - The data items input to a system must first be identified,
  - Processing required on the data items to produce the required outputs should be determined.

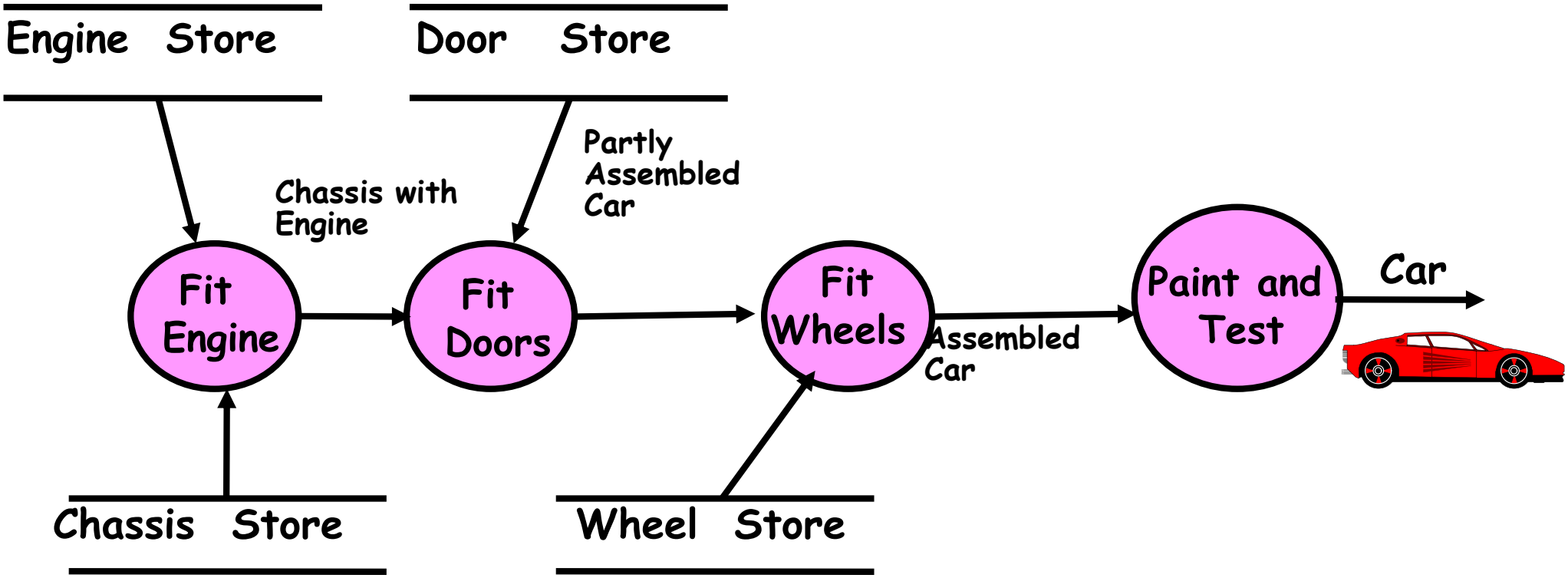
Data Flow-Oriented Design (Late 70s)

- Data flow technique identifies:
  - Different processing stations (functions) in a system.
  - The items (data) that flow between processing stations.

Data Flow-Oriented Design (Late 70s)

- Data flow technique is a generic technique:
  - Can be used to model the working of any system.
    - not just software systems.
- A major advantage of the data flow technique is its simplicity.

Data Flow Model of a Car Assembly Unit



Object-Oriented Design (80s)

- Object-oriented technique:
  - An intuitively appealing design approach:
  - Natural objects (such as employees, pay-roll-register, etc.) occurring in a problem are first identified.

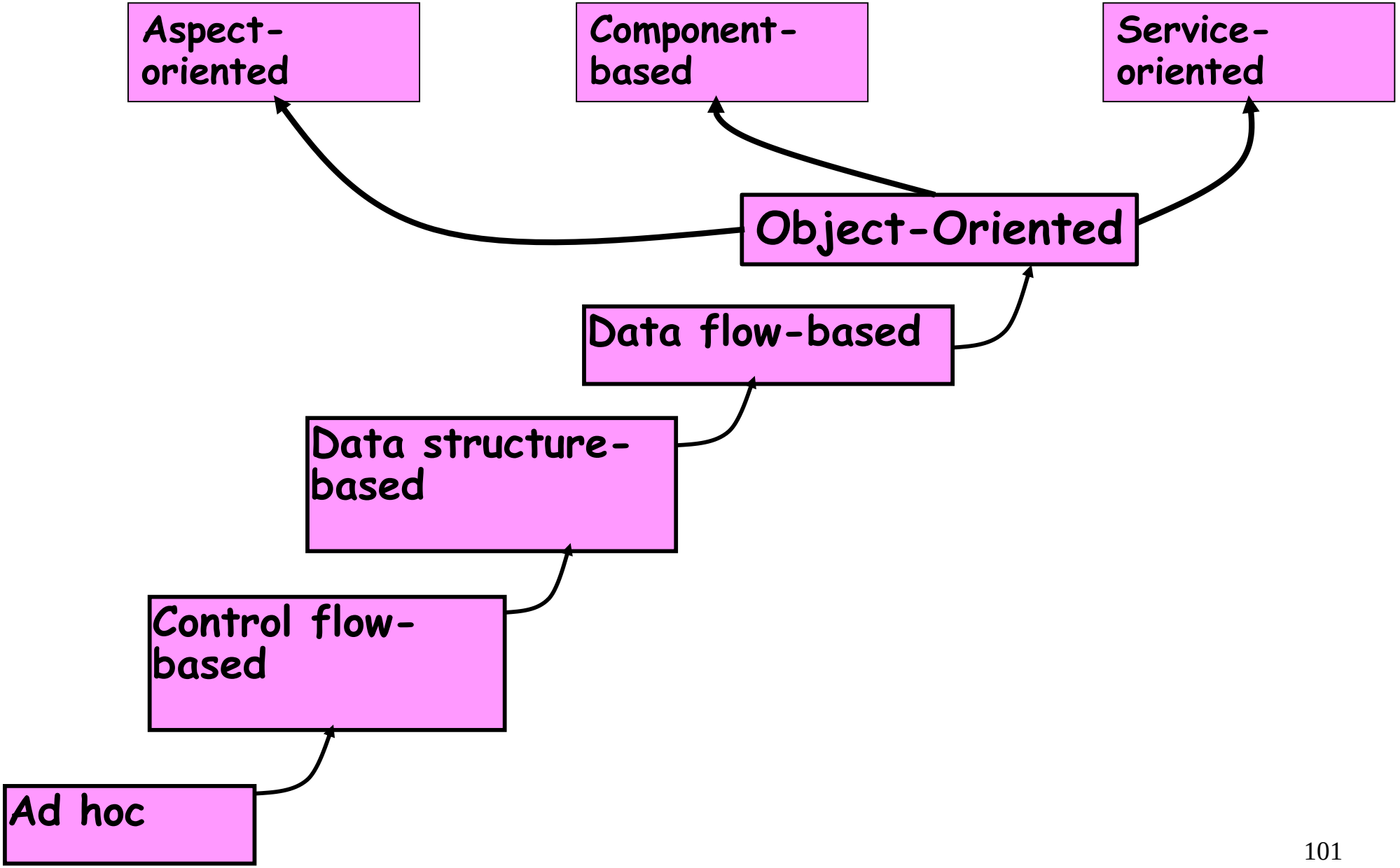
Object-Oriented Design (80s)

- Relationships among objects:
  - Such as composition, reference, and inheritance are determined.
- Each object essentially acts as:
  - A data hiding (or data abstraction) entity.

Object-Oriented Design (80s)

- Object-Oriented Techniques have gained wide acceptance:
  - **Simplicity**
  - **Increased Reuse possibilities**
  - **Lower development time and cost**
  - **More robust code**
  - **Easy maintenance**

Evolution of Design Techniques



Evolution of Other Software Engineering Techniques

- The improvements to the software design methodologies
  - are indeed very conspicuous.
- In additions to the software design techniques:
  - Several other techniques evolved.

Evolution of Other Software Engineering Techniques

- Life cycle models,
- Specification techniques,
- Project management techniques,
- Testing techniques,
- Debugging techniques,
- Quality assurance techniques,
- Metrics,
- CASE tools, etc.

Differences between the exploratory style and modern software development practices

- Use of Life Cycle Models
- Software is developed through several well-defined stages:
  - Requirements analysis and specification,
  - Design,
  - Coding,
  - Testing, etc.

Differences between the exploratory style and modern software development practices

- Emphasis has shifted
  - from error correction to error prevention.
- Modern practices emphasize:
  - detection of errors as close to their point of introduction as possible.

Differences between the exploratory style and modern software development practices (CONT.)

- In exploratory style,
  - errors are detected only during testing,
- Now:
  - Focus is on detecting as many errors as possible in each phase of development.

Differences between the exploratory style and modern software development practices (CONT.)

- In exploratory style:
  - coding is synonymous with program development.
- Now:
  - coding is considered only a small part of program development effort.

Differences between the exploratory style and modern software development practices (CONT.)

- A lot of effort and attention is now being paid to:
  - Requirements specification.
- Also, now there is a distinct design phase:
  - Standard design techniques are being used.

Differences between the exploratory style and modern software development practices (CONT.)

- During all stages of development process:
  - Periodic reviews are being carried out
- Software testing has become systematic:
  - Standard testing techniques are available.

Differences between the exploratory style and modern software development practices (CONT.)

- There is better visibility of design and code:
  - **Visibility means production of good quality, consistent and standard documents.**
  - In the past, very little attention was being given to producing good quality and consistent documents.
  - We will see later that increased visibility makes software project management easier.

Differences between the exploratory style and modern software development practices (CONT.)

- Because of good documentation:
  - fault diagnosis and maintenance are smoother now.
- Several metrics are being used:
  - help in software project management, quality assurance, etc.

Differences between the exploratory style and modern software development practices (CONT.)

- Projects are being prperly planned:
  - estimation,
  - scheduling,
  - monitoring mechanisms.
- Use of CASE tools.

Review Questions

- Graphically represent the important activities carried out in the build and fix style of program development.
  - What are the advantages and disadvantages of the build and fix style of software development?

Review Questions

- When using exploratory style, why does the development time and effort increase exponentially with linear increase in software size?
- What are the two basic mechanisms deployed by software engineering techniques to handle problem complexity?

Review Questions

- What is software crisis? What are its symptoms and cure?
- What is the difference between a programming job and assignment?
- What are the different types of software projects undertaken by software development companies?

Review Questions

- What is structured programming?
- What problems may appear if a large program is being developed without using structured programming techniques?